AvoLote — Guía de Sustentación y Defensa Técnica

Módulo de Usuarios, Catálogo de Aguacates (Hass y Papelillo) y Reservas en Tiempo Real • SENA 2026

Proyecto:	AvoLote (Catálogo y Gestión de Aguacate Hass y Papelillo)
Backend:	Django 6 + Django REST Framework + django-cors-headers + SQLite
Frontend:	Angular 21 (Signals, Standalone Components, Control Flow @if/@for) + TailwindCSS v4
Repositorio GitHub:	https://github.com/Diegosoprano777/proyectofinal
Backend en Render:	https://proyectofinal-pctw.onrender.com/api/
Usuarios Demo:	Admin: admin@sena.edu.co (admin123) • Cliente: cliente@sena.edu.co (cliente123)

1. Alcance y Objetivos de la Entrega

El proyecto resuelve la necesidad de los productores y comercializadores agrícolas de presentar su oferta de aguacates clasificados por variedad y estado de maduración, permitiendo a clientes minoristas o mayoristas apartar stock en tiempo real con sincronización garantizada mediante transacciones atómicas en el servidor.

- **Catálogo Dinámico:** Clasificación por variedad (Hass y Papelillo criollo) y estado de maduración (Verde, Pintón, Maduro) con fotografías asociadas.
- **Seguridad y Roles:** Autenticación vía token firmado con perfiles diferenciados (ADMIN y CLIENTE). El registro público sólo permite rol CLIENTE.
- **Control de Inventario Transaccional:** Resta inmediata de stock al apartar, impidiendo sobreventas mediante bloqueo con *select_for_update()*.
- **Ciclo de Vida de Apartados:** Estados PENDIENTE, ENTREGADO y CANCELADO, con devolución automática de kilogramos al lote al cancelar.

2. Arquitectura de Software y Tecnologías

Se adoptó un modelo desacoplado (Decoupled Single Page Application) que separa completamente la lógica de negocio en una API REST robusta y un cliente web reactivo de última generación:

Capa	Tecnología & Componentes	Responsabilidad
Frontend	Angular 21 + TailwindCSS v4 • Angular Signals (reactividad fina) • Control flow @if / @for • HttpInterceptor funcional	Experiencia de usuario moderna, filtros instantáneos sin recargas, gestión de sesión en localStorage e inyección automática de tokens Bearer.
Backend API	Django 6 + DRF • ModelSerializers con validaciones • TokenFirmadoAuthentication • Gunicorn + Whitenoise	Endpoints REST seguros, hashing de contraseñas con PBKDF2, reglas de autorización por rol y ejecución de transacciones ACID en base de datos.

Persistencia	SQLite con WAL Mode • UsuarioPersonalizado • ProductoLote • Apartado (con FK e integridad)	Almacenamiento relacional de usuarios, lotes con stock en kilogramos y registro histórico auditable de cada apartado.

3. Modelo Relacional y Reglas de Negocio

El modelo relacional se diseñó siguiendo estrictamente los principios de normalización e integridad referencial:

Entidades Principales:

- **UsuarioPersonalizado:** Contiene *nombre, correo (UK), password (hasheado)* y *rol ('ADMIN' | 'CLIENTE')*.
- **ProductoLote:** Contiene *codigo_lote (UK), variedad ('Hass' | 'Papelillo'), estado_maduracion ('VERDE' | 'PINTON' | 'MADURO'), precio_kg, stock_kg, dias_consumo* y *descripcion*.
- **Apartado:** Tabla intermedia relacional con claves foráneas a Usuario y Lote, guardando *cantidad_kg, precio_kg histórico, total en COP, estado ('PENDIENTE' | 'ENTREGADO' | 'CANCELADO')* y marcas de tiempo.

Reglas Críticas Implementadas en el Backend:

1. **Prevención de Elevación de Privilegios:** El serializador de registro marca el campo *rol* como solo lectura, asignando forzosamente *CLIENTE* a cualquier registro público.
2. **Prevención de Concurrencia y Sobreventa:** En *POST /api/catalogo/<pk>/apartar* se usa *select_for_update()* dentro de un bloque *transaction.atomic()*. Si la cantidad solicitada supera el stock, se rechaza con error 400 antes de mutar el registro.
3. **Devolución Automática de Stock:** Al cambiar el estado de un apartado a *CANCELADO*, el sistema devuelve exactamente los kilogramos apartados al lote correspondiente de forma atómica.

4. Matriz de Servicios REST (API Endpoints)

Método	Endpoint	Permiso	Descripción
POST	/api/usuarios/login	Público	Valida credenciales y genera token firmado de 12 horas.
POST	/api/usuarios/registro	Público	Registra nuevo cliente; previene asignación de ADMIN.
GET	/api/catalogo	Público	Lista lotes. Soporta ?estado_maduracion= y ?variedad=.
POST	/api/catalogo	Solo ADMIN	Crea un nuevo lote agrícola en el inventario.
PATCH	/api/catalogo/<id>	Solo ADMIN	Actualiza la maduración de un lote existente.
POST	/api/catalogo/<id>/apartar	Autenticado	Resta stock de forma atómica y crea registro de Apartado.
GET	/api/apartados	Autenticado	Cliente ve sus apartados; Admin visualiza todos con nombres.
PATCH	/api/apartados/<id>/estado	Autenticado	ENTREGADO (sólo admin) o CANCELADO (devuelve stock).

5. Guion Práctico de Sustentación (Demostración en 3 Minutos)

Sigue este orden exacto durante la sustentación para cubrir todos los criterios de evaluación de forma fluida y profesional:

Minuto 0:00 - 0:40 • Introducción y Arquitectura

> *"Buenos días profesor. Para el avance de hoy presentamos el módulo de Usuarios y Catálogo de Aguacates 'AvoLote'. Construimos una arquitectura totalmente desacoplada con Django REST Framework en el backend y Angular 21 con Signals y TailwindCSS v4 en el frontend. Ambas capas están desplegadas en la nube: la API en Render y el cliente en Vercel, conectadas vía HTTPS con CORS seguro."*

Minuto 0:40 - 1:40 • Demostración Rol CLIENTE

1. En la pantalla de login, señala los botones de acceso rápido demo y pulsa **'Entrar como Cliente'**.
2. Muestra la barra superior: indica que el sistema detectó el rol *CLIENTE* y sólo muestra las opciones correspondientes.
3. Demuestra los filtros reactivos: pulsa en **Papelillo** (muestra que las fotos cambian a la variedad criolla de cáscara lisa) y combina con el filtro **Maduro**.
4. En un lote con stock (ej: 400 kg), ingresa **15 kg** y haz clic en **'Apartar Aguacates'**. Señala la notificación toast de confirmación y cómo el stock bajó a 385 kg en vivo.
5. Abre el botón **'■ Mis apartados'**. Muestra el registro en estado ■ *Pendiente* con total calculado en pesos colombianos ($ COP).
6. Pulsa **'Cancelar'** en ese apartado. Explica al docente: *'Al cancelar, la base de datos ejecuta una transacción que retorna de inmediato los 15 kg al inventario para que otro comprador pueda adquirirlos'*.

Minuto 1:40 - 2:30 • Demostración Rol ADMINISTRADOR ('Papá')

1. Cierra sesión y pulsa el botón **'Entrar como Papá Admin'**.
2. Resalta que ahora se despliega el **Panel Rápido Admin** (protegido tanto en frontend como en los endpoints del backend).
3. Ve a la pestaña **'Cambiar maduración'**: selecciona un lote en estado Verde y cámbialo a Maduro. Observa cómo la tarjeta cambia de badge, color y fotografía al instante.
4. Abre **'■ Apartados'**: demuestra que el Admin visualiza todos los apartados globales con el nombre de cada cliente que solicitó el pedido, y que posee el botón **'✓ Entregar'** para despacharlo.

Minuto 2:30 - 3:00 • Pruebas Unitarias y Calidad

> *"Además de la interfaz visual, dejamos implementada una suite de 12 pruebas unitarias automatizadas en Django (python manage.py test api) que validan el 100% de los casos de negocio: prevención de sobreventa, no elevación de privilegios y aislamiento entre usuarios. Todo el proyecto se encuentra respaldado en GitHub con control de versiones."*

6. Respuestas a Preguntas Frecuentes del Evaluador

Pregunta Probable	Respuesta Técnica Sugerida
¿Cómo evitan que dos personas aparten el mismo stock a la vez?	Usamos transacciones atómicas en Django con *select_for_update()*. Esto bloquea la fila del lote en base de datos hasta que la resta de stock se confirme, evitando condiciones de carrera.
¿Por qué no usaron el User por defecto de Django?	El requerimiento solicitaba un modelo ligero con campos específicos (nombre, correo, rol). Usamos *UsuarioPersonalizado* con contraseñas cifradas mediante el algoritmo PBKDF2 de Django para mantener la seguridad.

¿Por qué usar Signals en Angular en vez de RxJS tradicional?	Angular Signals es el nuevo estándar oficial de Angular (zoneless-ready). Ofrece reactividad de grano fino, mejor rendimiento de renderizado y elimina fugas de memoria típicas de suscripciones manuales.
¿Qué pasos siguen para el proyecto final?	Migrar de SQLite a PostgreSQL en la nube, reemplazar tokens firmados por JWT con refresh tokens, permitir al Admin subir imágenes personalizadas por lote y generar reportes analíticos de ventas en PDF.